

SPRING WORKERS

Fruit Picking Synchronization Problem

Complex Engineering Problem (CEP) · Systems Programming

Course	Systems Programming
Language	C++20 (std::semaphore, std::thread)
Topic	Thread Synchronization with Semaphores
Test Cases	5 comprehensive scenarios

Submitted in partial fulfillment of course requirements

Contents

1	Problem Statement	2
1.1	Constraints	2
2	Solution Design	2
2.1	Threads	2
2.2	Synchronization Mechanism	2
2.3	Global Shared State	3
2.4	Control Flow	3
2.5	Key Synchronization Points Explained	3
2.5.1	Critical Section — Mutual Exclusion	3
2.5.2	Producer–Consumer: Picker → Loader	3
2.5.3	Termination Protocol	3
2.5.4	No Starvation	4
3	Source Code	5
4	Test Cases	6
4.1	Test Case 1 — 15 Fruits (Standard Operation)	6
4.2	Test Case 2 — Exactly 12 Fruits (Crate Fills Exactly)	6
4.3	Test Case 3 — Only 5 Fruits (Partial Crate, No Swap)	7
4.4	Test Case 4 — 0 Fruits (Empty Tree)	7
4.5	Test Case 5 — 24 Fruits (Two Full Crates)	7
4.6	Results Summary	9
5	Analysis and Discussion	10
5.1	Correctness	10
6	Conclusion	10

1. Problem Statement

The **Spring Workers** problem simulates a fruit orchard using concurrent threads. Three picker threads and one loader thread operate in parallel to harvest fruits from a tree and load them onto a truck.

Core Scenario

A tree holds a known number of fruits. Three pickers concurrently pick fruits and place them into a shared crate (capacity: 12 slots). When the crate is full, a picker calls the loader, waits for it to move the crate to the truck, and then resumes picking from a fresh empty crate. All pickers stop when the tree is bare. The loader places any partially filled crate into the truck at termination.

1.1 Constraints

- Each fruit is picked **exactly once** and by **exactly one picker** — no duplication.
- The crate is **shared** — only one picker may modify it at a time (mutual exclusion required).
- When a crate becomes full, the loader must swap it **before** any picker continues.
- The loader must wait for the current picker to complete its action before loading the crate.
- All pickers return once the tree is bare; the loader places any partial crate before exiting.
- Number of fruits and crate capacity are globally known.

2. Solution Design

2.1 Threads

The solution uses four C++ threads created via `std::thread`:

- `picker(1)`, `picker(2)`, `picker(3)` — three concurrent fruit-picking workers.
- `loader()` — one background thread managing crate-swapping and truck-loading.

2.2 Synchronization Mechanism

Three `binary_semaphore` objects (C++20 `<semaphore>`) coordinate the threads:

Semaphore	Initial Value	Purpose
<code>sem_pick</code>	1 (binary)	Mutual exclusion — one picker in the critical section at a time
<code>sem_load</code>	0 (binary)	Signals loader: crate is full (or all pickers done)
<code>sem_crate</code>	0 (binary)	Signals pickers: a fresh empty crate is ready

2.3 Global Shared State

- `fruits[]` — array of fruit names (the “tree”).
- `fruit_idx` — index of the next fruit to pick; protected by `sem_pick`.
- `CRATE[]` — shared array for the current crate; protected by `sem_pick`.
- `crate_idx` — current fill level of the crate.
- `TRUCK` — `vector<vector<string>>` of loaded crates.
- `active (atomic<int>)` — counts pickers still running.

2.4 Control Flow

Picker Thread	Loader Thread
<code>sem_pick.acquire() ← mutex</code>	<code>sem_load.acquire() ← block</code>
<code>if tree bare:</code>	<code>TRUCK.push_back(CRATE)</code>
<code>active--</code>	<code>crate_idx = 0</code>
<code>if last: sem_load.release()</code>	<code>if active==0: break</code>
<code>sem_pick.release(); break</code>	<code>sem_crate.release()</code>
<code>pick fruit → CRATE[crate_idx++]</code>	^
<code>if crate full:</code>	
<code>sem_load.release()</code>	>
<code>sem_crate.acquire() <</code>	
<code>sem_pick.release()</code>	

2.5 Key Synchronization Points Explained

2.5.1 Critical Section — Mutual Exclusion

`sem_pick` is acquired at the top of each picker’s loop and released at the bottom. This ensures only one picker reads `fruit_idx`, writes to `CRATE[]`, and increments `crate_idx` at a time — preventing race conditions such as two pickers placing the same fruit or writing to the same slot.

2.5.2 Producer–Consumer: Picker → Loader

When `crate_idx` reaches `CAPACITY`, the picker calls `sem_load.release()` while still inside the critical section (before releasing `sem_pick`). This wakes the loader with the crate state intact. The picker blocks on `sem_crate.acquire()`, preventing further picking until the loader confirms a fresh crate is available.

2.5.3 Termination Protocol

When a picker finds `fruit_idx >= FRUITS`, it decrements `active` and releases `sem_pick` to unblock the next picker. The **last** picker to exit (`active == 0`) releases `sem_load`, signaling the loader to load the final partial crate and exit cleanly.

2.5.4 No Starvation

`sem_pick` is binary and is released on every execution path (including `break`). No picker can starve another. The loader is woken exactly when needed — never spuriously.

3. Source Code

The full annotated source code is provided below as a screenshot from the IDE/compiler environment.

```
1  #include <iostream>
2  #include <thread>
3  #include <vector>
4  #include <semaphore>
5  #include <atomic>
6  #include <chrono>
7
8  using namespace std;
9
10 // GLOBAL VAR
11 binary_semaphore sem_pick(1);
12 binary_semaphore sem_load(0);
13 binary_semaphore sem_crate(0);
14
15 int fruit_idx = 0;
16 int crate_idx = 0;
17 atomic<int> active{3};
18
19 string fruits[30];
20 int FRUITS = 15;
21 int CAPACITY = 12;
22
23 string CRATE[12];
24 vector<vector<string>> TRUCK;
25
26 // Loader
27 void loader() {
28     while (true) {
29         sem_load.acquire();
30
31         if (crate_idx > 0) {
32             TRUCK.push_back(vector<string>(CRATE, CRATE + crate_idx));
33             cout << "[Loader] Loaded crate with " << crate_idx << " fruits\n";
34         }
35
36         crate_idx = 0;
37
38         if (active == 0) {
39             cout << "[Loader] Done! Truck has " << TRUCK.size() << " crates\n";
40             break;
41         }
42
43         sem_crate.release();
44     }
45 }
46
47 // Picker
48 void picker(int id) {
49     while (true) {
50         sem_pick.acquire();
51
52         if (fruit_idx >= FRUITS) {
53             active--;
54             if (active == 0) sem_load.release();
55             sem_pick.release();
56             break;
57         }
58
59         CRATE[crate_idx++] = fruits[fruit_idx];
60         cout << "[Picker-" << id << "] Picked " << fruits[fruit_idx]
61             << " (" << crate_idx << "/12)\n";
62         fruit_idx++;
63
64         if (crate_idx == CAPACITY) {
65             cout << "[Picker-" << id << "] Crate full!\n";
66             sem_load.release();
67             sem_crate.acquire();
68         }
69
70         sem_pick.release();
71         this_thread::sleep_for(chrono::milliseconds(30));
72     }
73     cout << "[Picker-" << id << "] Done\n";
74 }
75
76 vector<vector<string>> runner(const vector<string>& f, int cap = 12) {
77     TRUCK.clear();
78     fruit_idx = 0;
79     crate_idx = 0;
80     active = 3;
81     CAPACITY = cap;
82     FRUITS = (int)f.size();
83
84     for (int i = 0; i < FRUITS; ++i) {
85         fruits[i] = f[i];
86     }
87
88     thread t_loader(loader);
89     thread t1(picker, 1);
90     thread t2(picker, 2);
91     thread t3(picker, 3);
```

```

91     thread t3(picker, 3);
92
93     t1.join(); t2.join(); t3.join();
94     t_loader.join();
95
96     return TRUCK;
97 }
98
99 // Print Truck : just for printing nothing logical in it
100 void printTruck(const vector<vector<string>>& truck) {
101     if (truck.empty()) {
102         cout << " (Truck is empty)\n";
103         return;
104     }
105     for (size_t i = 0; i < truck.size(); ++i) {
106         cout << "Crate " << i+1 << " [" << truck[i].size() << "]: ";
107         for (const auto& fr : truck[i]) {
108             cout << fr << " ";
109         }
110         cout << "\n";
111     }
112 }
113
114
115 int main() {
116     cout << "=== Fruit Picking Simulation ===\n\n";
117
118     // Test Cases
119     vector<pair<string, vector<string>>> tests = {
120         {"15 Fruits (Standard)",
121          {"Apple", "Banana", "Mango", "Orange", "Grapes", "Pineapple", "Strawberry",
122           "Watermelon", "Papaya", "Guava", "Cherry", "Peach", "Pear", "Kiwi", "Blueberry"}},
123
124         {"Exact 12 Fruits",
125          {"Apple", "Banana", "Mango", "Orange", "Grapes", "Pineapple", "Strawberry",
126           "Watermelon", "Papaya", "Guava", "Cherry", "Peach"}},
127
128         {"Only 5 Fruits",
129          {"Apple", "Banana", "Mango", "Orange", "Grapes"}},
130
131         {"0 Fruits (Empty)", {}},
132
133         {"24 Fruits (Two Crates)",
134          {"Apple", "Banana", "Mango", "Orange", "Grapes", "Pineapple", "Strawberry",
135           "Watermelon", "Papaya", "Guava", "Cherry", "Peach", "Pear", "Kiwi",
136           "Blueberry", "Lemon", "Plum", "Apricot", "Lychee", "Fig", "Date",
137           "Pomelo", "Coconut", "Jackfruit"}}
138     };
139
140     for (auto& test : tests) {
141         cout << "TEST: " << test.first << " (" << test.second.size() << " fruits)\n";

```

4. Test Cases

Five test cases cover the full spectrum: standard operation, boundary conditions, edge cases, and multi-crate loads.

Following are these test cases we did perform

4.1 Test Case 1 — 15 Fruits (Standard Operation)

Field	Detail
Input	15 fruit names; crate capacity = 12
Expected	Crate 1: 12 fruits (full swap), Crate 2: 3 fruits (partial) → 2 crates
Rationale	Verifies primary workflow: one full crate triggers a loader swap, then the remaining 3 fruits form a second partial crate loaded at termination.

```
TEST: 15 Fruits (Standard) (15 fruits)
[Picker-1] Picked Apple (1/12)
[Picker-2] Picked Banana (2/12)
[Picker-3] Picked Mango (3/12)
[Picker-1] Picked Orange (4/12)
[Picker-3] Picked Grapes (5/12)
[Picker-2] Picked Pineapple (6/12)
[Picker-1] Picked Strawberry (7/12)
[Picker-3] Picked Watermelon (8/12)
[Picker-2] Picked Papaya (9/12)
[Picker-1] Picked Guava (10/12)
[Picker-2] Picked Cherry (11/12)
[Picker-3] Picked Peach (12/12)
[Picker-3] Crate full!
[Loader] Loaded crate with 12 fruits
[Picker-1] Picked Pear (1/12)
[Picker-2] Picked Kiwi (2/12)
[Picker-3] Picked Blueberry (3/12)
[Picker-1] Done
[Picker-2] Done
[Picker-3] Done
[Loader] Loaded crate with 3 fruits
[Loader] Done! Truck has 2 crates
Crate 1 [12]: Apple Banana Mango Orange Grapes Pineapple Strawberry Watermelon Papaya Guava Cherry Peach
Crate 2 [3]: Pear Kiwi Blueberry
→ Total Crates: 2
```


4.2 Test Case 2 — Exactly 12 Fruits (Crate Fills Exactly)

Field	Detail
Input	12 fruit names; crate capacity = 12
Expected	Crate 1: 12 fruits → 1 crate total
Rationale	Edge case: crate fills exactly at CAPACITY. Picker triggers sem_load after placing the 12th fruit. Loader swaps. Pickers then find the tree bare and terminate. Loader finds crate_idx == 0 and does not add an empty crate.

```
TEST: Exact 12 Fruits (12 fruits)
[Picker-1] Picked Apple (1/12)
[Picker-2] Picked Banana (2/12)
[Picker-3] Picked Mango (3/12)
[Picker-1] Picked Orange (4/12)
[Picker-2] Picked Grapes (5/12)
[Picker-3] Picked Pineapple (6/12)
[Picker-1] Picked Strawberry (7/12)
[Picker-2] Picked Watermelon (8/12)
[Picker-3] Picked Papaya (9/12)
[Picker-1] Picked Guava (10/12)
[Picker-2] Picked Cherry (11/12)
[Picker-3] Picked Peach (12/12)
[Picker-3] Crate full!
[Loader] Loaded crate with 12 fruits
[Picker-1] Done
[Picker-2] Done
[Picker-3] Done
[Loader] Done! Truck has 1 crates
Crate 1 [12]: Apple Banana Mango Orange Grapes Pineapple Strawberry Watermelon Papaya Guava Cherry Peach
→ Total Crates: 1
```

4.3 Test Case 3 — Only 5 Fruits (Partial Crate, No Swap)

Field	Detail
Input	5 fruit names; crate capacity = 12
Expected	Crate 1: 5 fruits → 1 crate total
Rationale	The crate never reaches CAPACITY, so sem_load is never triggered by a full crate. Instead the last picker (active == 0) triggers it at termination and the loader loads the partial crate.

```
TEST: Only 5 Fruits (5 fruits)
[Picker-1] Picked Apple (1/12)
[Picker-2] Picked Banana (2/12)
[Picker-3] Picked Mango (3/12)
[Picker-1] Picked Orange (4/12)
[Picker-2] Picked Grapes (5/12)
[Picker-3] Done
[Picker-1] Done
[Picker-2] Done
[Loader] Loaded crate with 5 fruits
[Loader] Done! Truck has 1 crates
Crate 1 [5]: Apple Banana Mango Orange Grapes
→ Total Crates: 1
```

4.4 Test Case 4 — 0 Fruits (Empty Tree)

Field	Detail
Input	Empty fruit list; crate capacity = 12
Expected	Truck is empty → 0 crates
Rationale	Stress-tests termination when the tree is already bare. All three pickers immediately exit. The last one triggers sem_load. Loader finds crate_idx == 0, skips loading, and exits cleanly.

```
TEST: 0 Fruits (Empty) (0 fruits)
[Picker-1] Done
[Picker-2] Done
[Picker-3] Done
[Loader] Done! Truck has 0 crates
(Truck is empty)
→ Total Crates: 0
```

4.5 Test Case 5 — 24 Fruits (Two Full Crates)

Field	Detail
Input	24 fruit names; crate capacity = 12
Expected	Crate 1: 12 fruits, Crate 2: 12 fruits → 2 crates total
Rationale	Verifies that the crate-swap cycle repeats correctly. First crate triggers swap at fruit 12; second crate triggers another at fruit 24. Pickers then find the tree bare. Loader exits with no partial crate remaining.

```
TEST: 24 Fruits (Two Crates) (24 fruits)
[Picker-1] Picked Apple (1/12)
[Picker-2] Picked Banana (2/12)
[Picker-3] Picked Mango (3/12)
[Picker-3] Picked Orange (4/12)
[Picker-2] Picked Grapes (5/12)
[Picker-1] Picked Pineapple (6/12)
[Picker-3] Picked Strawberry (7/12)
[Picker-2] Picked Watermelon (8/12)
[Picker-1] Picked Papaya (9/12)
[Picker-3] Picked Guava (10/12)
[Picker-2] Picked Cherry (11/12)
[Picker-1] Picked Peach (12/12)
[Picker-1] Crate full!
[Loader] Loaded crate with 12 fruits
[Picker-3] Picked Pear (1/12)
[Picker-2] Picked Kiwi (2/12)
[Picker-1] Picked Blueberry (3/12)
[Picker-2] Picked Lemon (4/12)
[Picker-3] Picked Plum (5/12)
[Picker-1] Picked Apricot (6/12)
[Picker-2] Picked Lychee (7/12)
[Picker-3] Picked Fig (8/12)
[Picker-1] Picked Date (9/12)
[Picker-2] Picked Pomelo (10/12)
[Picker-3] Picked Coconut (11/12)
[Picker-1] Picked Jackfruit (12/12)
[Picker-1] Crate full!
[Loader] Loaded crate with 12 fruits
[Picker-2] Done
[Picker-3] Done
[Picker-1] Done
[Loader] Done! Truck has 2 crates
Crate 1 [12]: Apple Banana Mango Orange Grapes Pineapple Strawberry Watermelon Papaya Guava Cherry Peach
Crate 2 [12]: Pear Kiwi Blueberry Lemon Plum Apricot Lychee Fig Date Pomelo Coconut Jackfruit
→ Total Crates: 2

All tests completed!
```


4.6 Results Summary

#	Test Name	Fruits	Expected Crates	Actual Crates
1	15 Fruits (Standard)	15	2	2
2	Exact 12 Fruits	12	1	1
3	Only 5 Fruits	5	1	1
4	0 Fruits (Empty)	0	0	0
5	24 Fruits (Two Crates)	24	2	2

5. Analysis and Discussion

5.1 Correctness

The solution guarantees the following properties:

- **No fruit picked twice** — `fruit_idx` is only incremented inside the critical section guarded by `sem_pick`.
- **No concurrent crate writes** — `crate_idx` is also modified only inside the critical section.
- **Loader woken exactly when needed** — either when the crate is full or when all pickers finish.
- **No empty crates in the truck** — the loader checks `crate_idx > 0` before pushing.

6. Conclusion

This CEP demonstrates the use of binary semaphores to solve a classic producer–consumer synchronization problem extended with mutual exclusion. The three-semaphore design cleanly separates the three coordination concerns:

- `sem_pick` — mutual exclusion over the shared crate.
- `sem_load` — synchronization between pickers and the loader on crate swaps.
- `sem_crate` — orderly continuation after a crate has been swapped.

All five test cases validate distinct scenarios — standard, boundary, edge, and multi-cycle — providing confidence in both correctness and robustness of the solution.

End of Report

